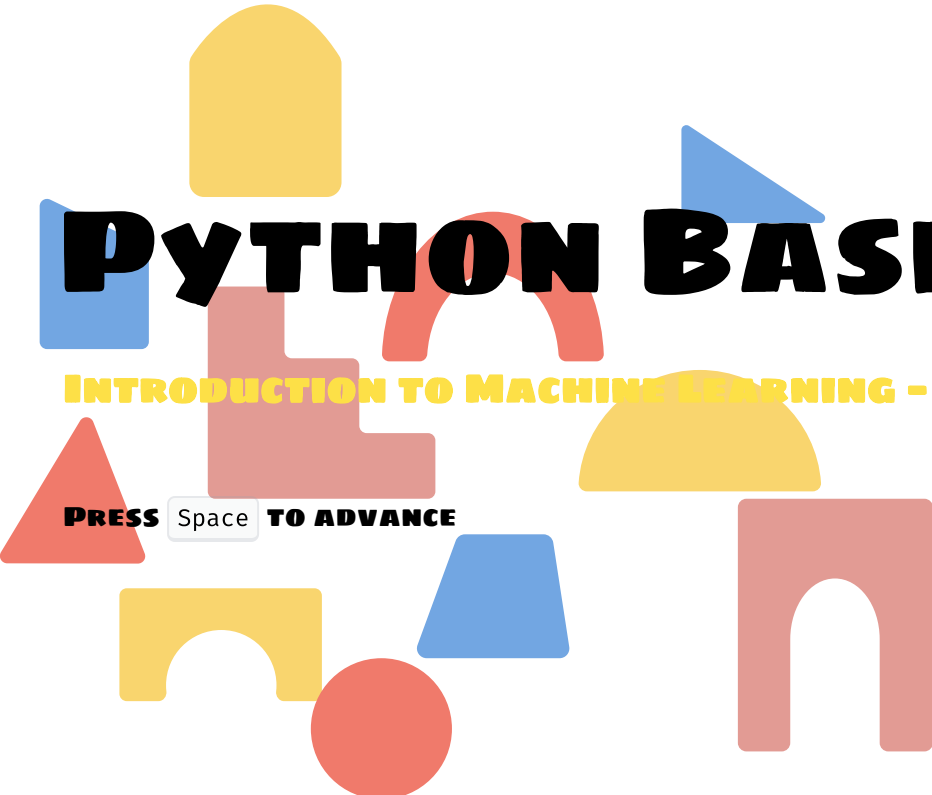


A collection of various geometric shapes including rectangles, triangles, circles, and semi-circles in yellow, blue, and red, scattered around the text.

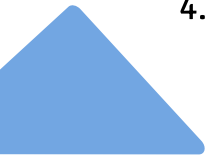
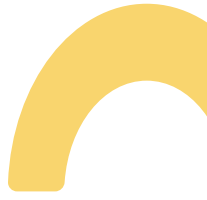
PYTHON BASICS

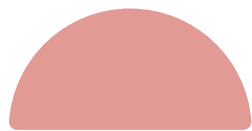
INTRODUCTION TO MACHINE LEARNING - LAB 01

PRESS Space **TO ADVANCE**



AGENDA

- 
- 
- 
1. Python Overview
 2. Setup & Execution Environments
 3. Variables & Built-in Types
 4. Control Flow
 5. Functions
 6. Classes
 7. Modules & Imports



PYTHON: AN OVERVIEW

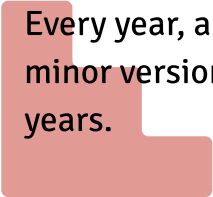




A BRIEF HISTORY

"Python is an experiment in how much freedom programmers need." — Guido van Rossum

- 
- **Late 1980s** — Guido van Rossum starts developing Python at CWI in the Netherlands
 - **February 1991** — Guido van Rossum releases Python 0.9.0
 - **January 1994** — Python 1.0: first official release
 - **October 2000** — Python 2.0: list comprehensions, garbage collection
 - **December 2008** — Python 3.0: breaking changes to fix design inconsistencies
 - Long coexistence period with Python 2 (EOL: **January 2020**)
 - **October 2025** — Python 3.14: latest stable release (at the time of writing)
 - **Today** — one of the most used languages in the world



Every year, around October, a new minor version is released with new features and optimizations. Each minor version is supported for around 18 months after release, and receives security updates for around 5 years.

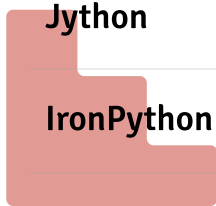




PYTHON IMPLEMENTATIONS

THE LANGUAGE SPEC CAN BE IMPLEMENTED IN DIFFERENT WAYS - EACH WITH DIFFERENT TRADE-OFFS.

Implementation	Language	Notes
CPython	C	Reference implementation. What you get from <code>python.org</code> . Most widely used.
PyPy	RPython	JIT-compiled. Up to 10× faster for CPU-bound loops. Does not support all features/extensions.
MicroPython	C	Lean subset of Python 3 for microcontrollers (ESP32, Raspberry Pi Pico).
Jython	Java	Runs on the JVM; largely unmaintained.
IronPython	C#	Runs on .NET; largely unmaintained.



KEY CHARACTERISTICS

High-level & interpreted No compilation step; code is executed line by line by the interpreter

Dynamically typed Variable types are resolved at runtime, not declared upfront

```
x = 42          # int
x = "hello"     # now a str - valid!
```

Strongly typed Types are *not* silently coerced — operations on incompatible types raise errors

```
"2" + 2 # TypeError!
```

Multi-paradigm Supports procedural, object-oriented, and functional styles

Garbage collected Memory is managed automatically via reference counting + cyclic GC

Batteries included Vast standard library: `os`, `math`, `json`, `http`, `re`, and hundreds more

Portable Runs on Windows, macOS, Linux, and many other platforms without modification

Why PYTHON FOR ML?

Language properties

- Simple, readable syntax → easy to learn
- Dynamic typing → fast prototyping
- REPL & notebooks → immediate feedback loop

Community

Largest ML/data-science community: tutorials, papers with code, pre-trained models, Stack Overflow answers, LLMs trained on Python code, etc.

Ecosystem

Library	Purpose
matplotlib / seaborn	Visualization
scikit-learn	Classical ML
numpy / jax	Fast array math
pytorch / tensorflow	Deep learning

Python as a glue language — Python itself is slow, but it doesn't need to be fast: its role is to *orchestrate* highly optimized libraries written in C, C++, or CUDA. When you call `numpy` or `pytorch`, the heavy computation runs outside the Python interpreter entirely.



SETUP & EXECUTION ENVIRONMENTS





THE PROBLEM WITH SYSTEM PYTHON

MOST OPERATING SYSTEMS SHIP WITH A PYTHON INSTALLATION USED BY SYSTEM TOOLS.

```
$ which python3
/usr/bin/python3          # system Python — hands off!
```

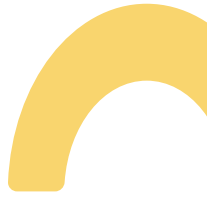


Installing packages globally causes two problems:

Dependency conflicts Project A needs
numpy=1.24 , project B needs numpy=2.0 .
Installing both globally is impossible.

System breakage Some OS tools depend on specific
package versions. A pip install gone wrong can
break your shell, package manager, or worse.

Rule of thumb: never pip install into your system Python. Always use a virtual environment.



VIRTUAL ENVIRONMENTS & VERSION MANAGEMENT

TWO ORTHOGONAL CONCERNS:

Concern	Question	Tools
Version management	Which Python version do I run?	<code>pyenv</code> , <code>uv</code>
Environment isolation	Which packages are installed?	<code>venv</code> , <code>conda</code> , <code>uv</code>

A virtual environment is just a directory containing:

- a copy (or symlink) of the Python interpreter
- its own folder for installed packages
- activation scripts that prepend it to `$PATH`

TOOLING LANDSCAPE

PYENV + VENV

The classic combo

```
pyenv install 3.12
pyenv local 3.12
python -m venv .venv
source .venv/bin/activate
pip install numpy
```

- ✓ Ships with stdlib (venv)
- ✗ Slow dependency resolver
- ✗ Two separate tools

CONDA / MAMBA

Popular in data science

```
conda create -n myenv python=3.12
conda activate myenv
conda install numpy
```

- ✓ All-in-one (Python + deps)
- ✓ Manages non-Python deps too
- ✗ Heavy install (Anaconda)
- ✗ Separate ecosystem from PyPI

See [pixi](#) for a modern take on this approach.

UV ★

Modern all-in-one tool

```
uv init myproject --python 3.12
cd myproject
uv add numpy
uv run python script.py
```

- ✓ All-in-one (Python + deps)
- ✓ 10–100× faster than pip
- ✓ Both a package manager and project manager

GETTING STARTED WITH uv

uv IS OUR RECOMMENDED TOOL FOR THIS COURSE.

The fastest way to install it is via the official install script (see [uv docs](#) for more options):

```
# Linux / macOS:
curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows:
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

uv covers two distinct use cases — pick whichever fits your situation:

Mode	Replaces	Best for
Package manager	pyenv + venv + pip	quick scripts, course exercises
Project manager	poetry , hatch	real projects with lockfiles & packaging



uv AS A PACKAGE MANAGER

DROP-IN REPLACEMENT FOR `pyenv` + `venv` + `pip` - **NO** `pyproject.toml` **NEEDED.**

```
# Install a specific Python version
uv python install 3.12

# Create a virtual environment
uv venv --python 3.12          # creates .venv/ in the current directory
source .venv/bin/activate

# Install packages (same interface as pip)
uv pip install numpy pandas matplotlib

# Sync from a requirements file
uv pip install -r requirements.txt
```

All `uv pip` commands are identical to `pip` but significantly faster. Use this mode when you just need a quick, isolated environment.



uv AS A PROJECT MANAGER

MANAGES THE FULL PROJECT LIFECYCLE - DEPENDENCIES, LOCKFILE, PYTHON VERSION - VIA `pyproject.toml` .

```
# Create a new project
uv init myproject --python 3.12 && cd myproject

# Add / remove dependencies (updates pyproject.toml + uv.lock)
uv add numpy polars matplotlib
uv remove pandas

# Reproduce the exact environment on another machine
uv sync                # reads uv.lock
```

Project layout:

```
myproject/
├── .venv/           # virtual environment (don't commit)
├── pyproject.toml   # project metadata & dependencies
├── uv.lock          # exact pinned versions (commit this!)
├── .python-version  # Python version (optional)
└── hello.py
```

EXECUTING PYTHON: THE REPL

REPL = READ-EVAL-PRINT LOOP. AN INTERACTIVE SHELL FOR QUICK EXPERIMENTS.

```
$ python          # or: uv run python
Python 3.12.3
>>> 1 + 1
2
>>> name = "ML"
>>> f"Hello, {name}!"
'Hello, ML!'
>>> exit()
```

Useful for:

- Testing a quick expression
- Exploring an object's API (`dir(obj)` , `help(obj)`)
- Debugging a single value

IPython (`uv pip install ipython`) gives you a much nicer REPL: tab completion, syntax highlighting, `%magic` commands, and persistent history.

EXECUTING PYTHON: JUPYTER NOTEBOOKS

NOTEBOOKS EXTEND THE REPL IDEA - CELLS CAN BE RUN IN ANY ORDER, AND OUTPUT (INCLUDING PLOTS) IS SAVED ALONGSIDE THE CODE.

```
uv pip install jupyter
jupyter lab    # opens in the browser
```

Pros

- Immediate visual feedback (plots inline)
- Mix code with Markdown explanations
- Easy to share as a self-contained document
- Standard format in ML research & courses

Cons / pitfalls

- Hidden state: cells can run out of order
- Poor version control (JSON format)
- Hard to test and refactor
- Encourages global scope

Use notebooks for **exploration and communication**. Move mature, reusable code into `.py` files.

EXECUTING PYTHON: SCRIPTS & PROJECTS

AS SOON AS CODE GROWS BEYOND A NOTEBOOK, YOU MOVE TO `.py` FILES.

```
$ python script.py
# or without activating the environment:
$ uv run python script.py
```

Single script A self-contained `.py` file — good for automation, data pipelines, small tools.

```
my_script.py
```

Multi-file project Code split across modules, each responsible for one concern — the natural end point as a project grows.

```
myproject/
├── main.py
├── data.py
├── model.py
└── utils.py
```

We will come back to how to structure multi-file projects in the **Modules & Imports** section.

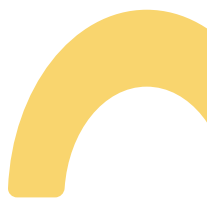


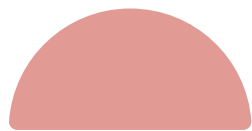
VISUAL STUDIO CODE

WE RECOMMEND USING VS CODE AS YOUR CODE EDITOR FOR THIS COURSE. IT'S FREE, "OPEN-SOURCE", AND HAS EXCELLENT PYTHON SUPPORT.

Extension	What it does
Python	IntelliSense, debugger, test runner, env selection
Jupyter	Edit and run notebooks inside VS Code (with support for Colab notebooks too)
ruff	Extremely fast linter + formatter
basedpyright/ty/pyrefly	Type checking and code intelligence (optional)

Select your interpreter — open the Command Palette (`Ctrl+Shift+P`) → *Python: Select Interpreter* → pick the `.venv` inside your project.





VARIABLES & BUILT-IN TYPES



VARIABLES & ASSIGNMENT

PYTHON VARIABLES ARE NAMES THAT REFER TO OBJECTS IN MEMORY.

```
x = 42                # bind x to the integer 42
x = "hello"           # rebind x to a string – perfectly valid

a, b, c = 1, 2, 3     # multiple assignment (tuple unpacking)
a, b = b, a           # swap without a temp variable

x = y = 0             # bind multiple names to the same object (modification via one name affects all)
```

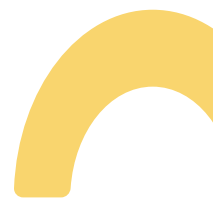
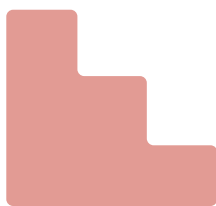
Naming rules

- Must start with a letter or `_` ; can contain letters, digits, `_`
- Case-sensitive (`count` $\neq$ `Count`)
- Convention: `snake_case` for variables and functions, `UPPER_CASE` for constants

SCALAR TYPES AT A GLANCE



Type	Example	Notes
<code>int</code>	<code>42</code> , <code>-7</code> , <code>0xFF</code> , <code>0b1010</code>	Arbitrary precision — no overflow
<code>float</code>	<code>3.14</code> , <code>1e-5</code> , <code>float('inf')</code>	IEEE 754 double precision
<code>complex</code>	<code>2+3j</code>	Built-in, rarely needed in ML
<code>bool</code>	<code>True</code> , <code>False</code>	Subclass of <code>int</code> : <code>True = 1</code> , <code>False = 0</code>
<code>NoneType</code>	<code>None</code>	The null / "no value" sentinel



NUMBERS

```
# int - arbitrary precision
2 ** 100          # 1267650600228229401496703205376 (no overflow!)

# float - IEEE 754 double
0.1 + 0.2         # 0.30000000000000004
round(0.1 + 0.2, 2) # 0.3

# Arithmetic operators
10 / 3            # 3.3333...   true division → always float
10 // 3           # 3           floor division → int
10 % 3            # 1           modulo
2 ** 8            # 256         exponentiation

# Useful builtins (more in the math module)
abs(-5)           # 5
round(3.567, 2)   # 3.57
```



bool AND COMPARISON OPERATORS

bool **IS A SUBCLASS OF** int - True **BEHAVES AS** 1, False **AS** 0.

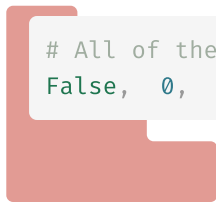
```
# Comparison operators – always return a bool
3 > 2          # True          5 < 3          # False
3 == 3         # True          3 != 3         # False
3 >= 3         # True          "a" < "b"    # True (lexicographic)

# Logical operators
True and False # False – short-circuits: skips right side if left is False
True or False  # True  – short-circuits: skips right side if left is True
not True       # False

# bool is a subclass of int – useful in practice
True + True    # 2
sum([True, False, True, True]) # 3 ← count how many conditions hold
```

Falsy values – everything else is truthy

```
# All of these evaluate to False in a boolean context:
False, 0, 0.0, "", [], {}, set(), None
```





None

None **IS PYTHON'S NULL VALUE - THE SINGLE INSTANCE OF** `NoneType` .

```
result = None           # explicit "no value yet"
```

Always check for `None` with `is` , not `=`

`is` and `is not` check for identity (same object in memory), while `=` checks for equality. Since `None` (like `True` and `False`) is a singleton, using `is` is the correct way to check for it.

```
# Correct
if result is None:
    print("nothing found")

if result is not None:
    process(result)

# Wrong - a custom class could define __eq__ to return True when compared to None
if result = None:    # ← avoid
    ...
```



LISTS

ORDERED, HETEREGENOUS, MUTABLE SEQUENCES.

```
nums = [3, 1, 4, 1, 5, 9]
mixed = [1, "two", 3.0, True]  # can hold any types

# Mutation
nums.append(2)                # add one element to the end
nums.insert(0, 0)             # insert value at index
nums.extend([6, 5])           # append all elements from another iterable
nums.pop()                    # remove & return the last element
nums.pop(2)                   # remove & return element at index 2
nums.remove(1)                 # remove the first occurrence of the value
nums.sort()                   # sort in place (modifies the list)
sorted(nums)                  # returns a new sorted list (original unchanged)
len(nums)                     # number of elements

# Concatenation & repetition
[1, 2] + [3, 4]               # [1, 2, 3, 4]
*[1, 2], 7, 8]               # [1, 2, 7, 8] (unpacking operator)
[0] * 5                       # [0, 0, 0, 0, 0]
[[1, 2]] * 3                  # [[1, 2], [1, 2], [1, 2]] (shallow copy – beware of mutability)
```

INDEXING & SLICING

APPLIES TO ALL ORDERED SEQUENCES: `str`, `list`, `tuple`.

```
s = [ 3,  1,  4,  1,  5,  9]
index:  0   1   2   3   4   5      (forward – from 0)
        -6  -5  -4  -3  -2  -1      (backward – from -1)
```

```
s[0]      # 3      first element
s[-1]     # 9      last element
s[-2]     # 5      second to last
s[100]    # IndexError: list index out of range
```

Slicing `[start : stop : step]` – returns a new sequence, `stop` is excluded

```
s[1:4]     # [1, 4, 1]    indices 1, 2, 3
s[:3]      # [3, 1, 4]    from the beginning
s[3:]      # [1, 5, 9]    to the end
s[::2]     # [3, 4, 5]    every other element
s[::-1]    # [9, 5, 1, 4, 1, 3] reversed
s[3:100]   # [1, 5, 9]    slicing is forgiving – no error if start/stop out of bounds
s[4:2]     # []           empty slice if start > stop (with positive step)
```

TUPLES

ORDERED, HETEREGENOUS, IMMUTABLE SEQUENCES.

```
point = (3, 7)
rgb = (255, 128, 0)
single = (42,)      # trailing comma required for a 1-element tuple

# Indexing – same as lists
point[0]      # 3
rgb[0:2]      # (255, 128)

# Mutation – not allowed! Tuples are immutable
rgb[0] = 0    # TypeError: 'tuple' object does not support item assignment
del rgb[1]    # TypeError: 'tuple' object does not support item deletion

# Unpacking
x, y = point
r, g, b = rgb
first, *rest = (1, 2, 3, 4)  # first=1, rest=[2, 3, 4]
```

STRINGS

STRINGS ARE IMMUTABLE SEQUENCES OF UNICODE CHARACTERS.

```
s = "Hello, world!"

# Common methods (always return new strings – strings are immutable)
s.upper()           # 'HELLO, WORLD!'
s.replace("world", "Python") # 'Hello, Python!'
s.split(", ")       # ['Hello', 'world!']
" padded ".strip()  # 'padded'
", ".join(["a", "b", "c"]) # 'a, b, c'

# Indexing & slicing – same as lists
s[0]           # 'H' this is still a string, not a character type
s[7:12]        # 'world' substring
s[0] = "h"     # TypeError: 'str' object does not support item assignment
```

f-strings (Python 3.6+) – the preferred way to format strings

```
name, score = "Alice", 9.81
f"Student: {name}, score: {score:.2f}" # 'Student: Alice, score: 9.81'
f"{2 ** 10} = {" # '2 ** 10 = 1024' (debug syntax)
```

SETS

UNORDERED COLLECTIONS OF UNIQUE ELEMENTS. BACKED BY A HASH TABLE.

```
a = {1, 2, 3, 2, 1}    # {1, 2, 3} - duplicates removed automatically
empty = set()          # not {} - that creates an empty dict!
```

```
# Mutation
```

```
a.add(4)
```

```
a.discard(99)  # remove if present, no error if missing
```

```
# Set operations
```

```
b = {3, 4, 5, 6}
```

```
a | b    # {1, 2, 3, 4, 5, 6}  union
```

```
a & b    # {3, 4}              intersection
```

```
a - b    # {1, 2}              difference
```

```
a ^ b    # {1, 2, 5, 6}        symmetric difference
```

Key property: membership test is $O(1)$ — much faster than lists for large collections.

```
3 in {1, 2, 3, 4, 5}    # True - instant (hash lookup)
6 not in [1, 2, 3, 4, 5] # True -  $O(n)$  scan
```

DICTIONARIES

KEY → VALUE MAPPINGS. ORDERED BY INSERTION ORDER SINCE PYTHON 3.7.

```
student = {"name": "Alice", "score": 9.5, "passed": True} # heterogeneous keys and values allowed

# Access
student["name"]          # 'Alice' - KeyError if missing
student.get("grade", "N/A") # 'N/A' - safe access with default

# Mutation
student["score"] = 10.0   # update existing key
student["year"] = 2       # add new key
del student["passed"]     # remove key

# Useful methods
student.keys()            # dict_keys(['name', 'score', 'year'])
student.values()          # dict_values(['Alice', 10.0, 2])
student.items()           # dict_items([('name', 'Alice'), ...])
```

THE MUTABILITY GOTCHA (REFERENCES)

VARIABLES ARE POINTERS TO OBJECTS. ASSIGNMENT NEVER COPIES DATA!

```
a = [1, 2, 3]
b = a          # 'b' now points to the EXACT SAME list as 'a'

b.append(4)
print(a)       # [1, 2, 3, 4] ← 'a' is modified too!
```

How to actually copy a list:

```
# 1. Using the .copy() method
c = a.copy()

# 2. Using a full slice
d = a[:]

# 3. Using the list() constructor
e = list(a)

d.append(5)
print(a)      # [1, 2, 3, 4] (unchanged)
```

HASHABILITY & (IM)MUTABILITY

To look up keys instantly, `set` and `dict` compute a mathematical "hash" of the object. Since the hash must never change during the object's lifetime, only immutable types are hashable by default.

```
hash(42)          # 42
hash("hello")     # 974541110297264300
hash((1, 2, 3))   # 529344067295497451
hash([1, 2, 3])   # TypeError: unhashable type: 'list'
```

Cheat Sheet:

- Hashable (can be keys/sets): `int`, `float`, `bool`, `str`, `tuple`, `NoneType`
- Unhashable (cannot be keys/sets): `list`, `dict`, `set`

Note: A tuple is only hashable if all its contents are also hashable. `(1, 2, "a")` is fine, but `(1, 2, [3, 4])` will raise a `TypeError` !

TYPE CHECKING & CONVERSION

```
# Check the type of an object
type(42)           # <class 'int'>
type("hello")     # <class 'str'>

# isinstance – preferred over type(): handles inheritance
isinstance(True, bool)      # True
isinstance(True, int)       # True (bool IS an int!)
isinstance(42, (int, float)) # True – accepts a tuple of types
```

Explicit conversion (casting)

```
int("42")         # 42
int(3.9)           # 3      – truncates, does NOT round
float("3.14")     # 3.14
str(100)           # '100'
bool(0)            # False
bool("False")     # True   – non-empty strings are truthy!
list((1,2,3))     # [1, 2, 3]
tuple([1,2,3])    # (1, 2, 3)
set([1,1,2,3])    # {1, 2, 3}
```

TYPING SYSTEMS: DYNAMIC & STRONG

Strong (Strict: No implicit coercion) Weak (Flexible: Silent type coercion)

Static

(Checked at compile-time)

Java, Rust, C#

C, C++

Dynamic

(Checked at runtime)

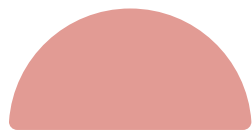
Python, Ruby

JavaScript, PHP

Strong vs. Weak in Action: Python refuses to guess, while JS silently coerces types.

```
# Python (Dynamic + Strong)
"5" + 3      # TypeError (no implicit string cast)
5 + "3"      # TypeError (no implicit int cast)
5 - "3"      # TypeError
"5" * 3      # '555' (defined: string repetition)
```

```
// JavaScript (Dynamic + Weak)
"5" + 3      // "53" (coerces 3 to string)
5 + "3"      // "53" (coerces 5 to string)
5 - "3"      // 2   (coerces "3" to number)
"5" * 3      // 15  (coerces "5" to number)
```



CONTROL FLOW



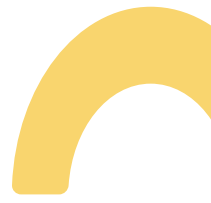


Why CONTROL FLOW?

By default, Python runs scripts top-to-bottom, line by line.

Control flow statements allow your program to react to data by:

- 
- **Making decisions:** Run code *only if* a condition is met.
 - **Branching logic:** Handle different scenarios dynamically.
 - **Repeating actions:** Run the same block of code multiple times.





if / elif / else

USED TO CONDITIONALLY EXECUTE BLOCKS OF CODE BASED ON BOOLEAN EXPRESSIONS.

Python uses **indentation** (4 spaces by convention - PEP 8) to delimit blocks — no braces.

```
x = 42

if x > 100:
    print("large")
elif x > 10:
    print("medium")    # this branch runs
else:
    print("small")
```

Inline (ternary) form — for simple conditional expressions, not statements

```
label = "even" if x % 2 == 0 else "odd"
```



BLOCKS & INDENTATION

UNLIKE MANY LANGUAGES, PYTHON DOES NOT USE BRACES `{ }` TO GROUP CODE. IT STRICTLY RELIES ON WHITESPACE (INDENTATION).

```
x = 10

if x > 5:
    print("This is inside the block")

    if x == 10:
        print("Blocks can be nested!")

print("This is outside the block")
```

Key points:

- Use **4 spaces** per indentation level (never mix spaces and tabs!).
- Inconsistent indentation will crash your program with an `IndentationError`.
- A block ends simply when the indentation returns to the previous level.

VARIABLE SCOPING

PYTHON DOES NOT HAVE BLOCK-LEVEL SCOPING!

Variables created inside an `if`, `match`, `while`, or `for` block **spill out** and remain accessible after the block finishes.

```
if True:
    secret_message = "I leaked out!"

# In C++ or Java, this would be an error. In Python, it works perfectly:
print(secret_message)
```

But be careful — if the block never runs, the variable is never created, leading to a `NameError` :

```
x = []
if len(x) > 0: # could be written as "if x:"
    first = x[0]

print(first) # NameError: name 'first' is not defined
```



match / case (PYTHON 3.10+)

USED FOR PATTERN MATCHING - REPLACES COMPLEX if / elif CHAINS

```
match status_code: # status_code is an integer
    case 200:
        print("OK")
    case 400 | 404: # Pipe (|) acts as a logical OR
        print("Client Error")
    case _:
        # Underscore (_) is the default/fallback
        print("Unknown code")
```

Structural pattern matching — match against data structures, not just values

```
match point: # point is a tuple (x, y)
    case (0, 0):
        print("Origin")
    case (x, 0) if x > 0: # guard condition
        print(f"Positive X-axis at {x}")
    case (0, y):
        print(f"Y-axis at {y}")
    case (x, y):
        print(f"Point at ({x}, {y})")
```





while **LOOPS**

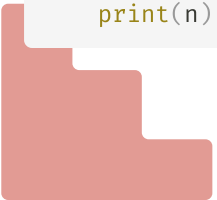
REPEAT A BLOCK AS LONG AS A CONDITION IS True .

```
n = 1
while n < 100:
    n *= 2

print(n)    # 128
```

You can alter the flow inside the loop using **break** and **continue** .

```
n = 0
while n < 5:
    n += 1
    if n == 3:
        continue # skips the rest of the block, jumps to next iteration
    if n == 5:
        break    # exits the loop immediately entirely
    print(n)     # Output: 1, 2, 4
```



while **LOOPS**

To create **do-while** loops, use a **while True** with a **break** condition inside.

```
while True:
    command = input("Enter a command: ")
    print(f"You entered: {command}")
    if command == "exit":
        break
```

Else clause — runs if the loop completes without hitting a **break**

```
x = [1, 2, 3]
i = 0
while i < len(x):
    if x[i] < 0:
        print("Negative number found!")
        break
    i += 1
else:
    print("All numbers are non-negative.")
```



for **LOOPS** & range()

for **ITERATES OVER ANY ITERABLE - LISTS, STRINGS, DICTS, FILES, GENERATORS, ...**

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)

# Iterating over a string
for ch in "ML":
    print(ch)    # 'M', then 'L'

# Iterating over a dict (keys by default)
d = {"a": 1, "b": 2}
for key in d:
    print(key)   # 'a', then 'b'

for value in d.values():
    print(value) # 1, then 2

for key, value in d.items():
    print(key, value) # 'a' 1, then 'b' 2
```





for **LOOPS & HELPER FUNCTIONS**

PYTHON PROVIDES BUILT-IN FUNCTIONS TO MAKE COMMON LOOP PATTERNS EASIER:

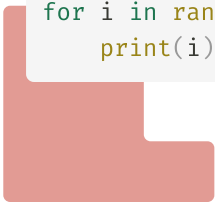
```
# Looping with indices
for i, fruit in enumerate(fruits):
    print(i, fruit) # 0 apple, 1 banana, 2 cherry

# Loop over multiple iterables in parallel
names = ["Alice", "Bob", "Charlie"]
for name, fruit in zip(names, fruits): # use strict=True to enforce same length
    print(name, fruit) # Alice apple, Bob banana, Charlie cherry

# Loop over a range of numbers
for i in range(5): # no need to create a list of numbers
    print(i) # 0, 1, 2, 3, 4

for i in range(2, 10): # start at 2, end before 10
    print(i) # 2, 3, 4, 5, 6, 7, 8, 9

for i in range(0, 20, 5): # step by 5
    print(i) # 0, 5, 10, 15
```



LIST COMPREHENSIONS

A FASTER AND CONCISE, READABLE WAY TO BUILD LISTS - PREFERRED OVER

map / filter IN PYTHON.

```
squares = []
for x in range(10):
    squares.append(x ** 2) # this is slow (we keep resizing the list) and verbose

# Basic form: [expression for item in iterable]
squares = [x ** 2 for x in range(10)] # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# With a filter:[expression for item in iterable if condition]
even_squares = [x ** 2 for x in range(10) if x % 2 == 0] # [0, 4, 16, 36, 64]
```

Dict and Set comprehensions use `{}` :

```
word_lengths = {w: len(w) for w in ["cat", "elephant"]} # {'cat': 3, 'elephant': 8}
unique_remainders = {x % 5 for x in range(20)}          # {0, 1, 2, 3, 4}
```

Rule of thumb: Keep comprehensions to one logical transformation. If it gets too complex, use a standard `for` loop!

GENERATOR EXPRESSIONS

LIKE LIST COMPREHENSIONS BUT LAZY - VALUES ARE PRODUCED ON DEMAND, NOT STORED.

```
# List comprehension - builds the entire list in memory
squares_list = [x ** 2 for x in range(10_000_000)] # ~89 MB

# Generator expression - produces values one at a time
squares_gen = (x ** 2 for x in range(10_000_000)) # ~200 bytes
```

Generators can be consumed by any function that accepts an iterable:

```
total = sum(x ** 2 for x in range(1000)) # no intermediate list created, memory efficient
big    = any(x > 999 for x in range(10_000)) # stops as soon as True, the rest is never produced
```

Note: Generators are single-use and do not support indexing or slicing.

```
gen = (x ** 2 for x in range(5))
x = sum(gen) # consumes the generator
y = sum(gen) # now it's empty, so y is 0
```

```
gen = (x ** 2 for x in range(5))
print(gen[0]) # TypeError
print(gen[:3]) # TypeError
```



RAISING EXCEPTIONS

NOT ALWAYS OUR CODE CAN HANDLE EVERY SITUATION. SOMETIMES WE NEED TO SIGNAL THAT SOMETHING WENT WRONG OR THAT A CONDITION IS NOT MET.

You can raise exceptions manually to signal errors or invalid conditions:



```
x = int(input("Enter a positive number: "))
if x < 0:
    raise ValueError("Negative number is not allowed!")
```

This will stop the normal flow of the program and print a traceback, unless the exception is caught by a `try / except` block (see next slide).

You can also define your own custom exceptions by subclassing `Exception` (we will see classes later, but this is just to show the syntax):

```
class MyCustomError(Exception):
    pass

# then raise it like this:
raise MyCustomError("Something specific went wrong!")
```





ERROR HANDLING

WHEN YOU WANT TO HANDLE POTENTIAL ERRORS GRACEFULLY, YOU CAN USE `try / except` BLOCKS TO CATCH EXCEPTIONS AND PREVENT YOUR PROGRAM FROM CRASHING.

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

You can catch multiple exceptions:

```
try:
    x = int(input("Enter a number: "))
    result = 10 / x
except ValueError:
    print("That's not a valid number!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
except: # catch-all (not recommended, use only for logging or re-raising)
    print("An unexpected error occurred.")
    raise # re-raise the exception after handling/logging
```



ERROR HANDLING

YOU CAN ALSO USE `else` AND `finally` CLAUSES WITH `try / except :`

```
try:
    x = int(input("Enter a number: "))
except ValueError:
    print("That's not a valid number!")
else:
    # this block runs only if no exception was raised in the try block
    print(f"You entered {x}, which is valid.")
finally:
    # this block runs no matter what, even if an exception was raised and not caught
    print("This will always run, even if an error occurred.")
```

When to use `else` and `finally` :

- Use `else` for code that should only run if the `try` block succeeded without exceptions (you will rarely need this)
- Use `finally` for cleanup code that must run regardless of success or failure (e.g., closing files, releasing resources/locks/database connections, etc.)



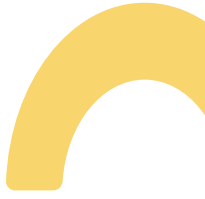
FUNCTIONS





Why Use Functions?

FUNCTIONS ARE THE BUILDING BLOCKS OF READABLE AND MAINTAINABLE CODE.

- **Reusability (DRY):** *Don't Repeat Yourself*. Write a logic once, use it everywhere.
 - **Modularity:** Break down complex pipelines into smaller, manageable chunks (e.g., `load_data()`, `train_model()`).
 - **Abstraction:** Hide complex implementation details behind a simple name.
 - **Testing:** Small, isolated functions are much easier to test and debug.
- 
- 
- 

DEFINING FUNCTIONS

DEFINED WITH `def` AND RETURN VALUES WITH `return`.

```
def add(a, b):  
    return a + b  
  
result = add(3, 5)    # 8
```

- A function without a `return` implicitly returns `None`.
- `return` stops the function immediately (useful for early exits).

```
def sign(x):  
    if x > 0:  
        return "positive"  
  
    # if x is greater than 0, we won't reach this point  
    if x < 0:  
        return "negative"  
  
    return "zero"
```

RETURNING MULTIPLE VALUES

PYTHON DOES NOT PUT ANY CONSTRAINTS ON WHAT YOU CAN RETURN.

Returning multiple values is common and is done by returning a tuple:

```
def min_max(values):  
    # Actually returns a tuple: (min, max)  
    return min(values), max(values)
```

```
# Tuple unpacking  
lo, hi = min_max([3, 1, 4, 1, 5])
```

A function can return values of different types:

```
def never_do_this(x):  
    if x < 0:  
        return "negative"  
    elif x > 0:  
        return 1  
    # if x is zero, we return None implicitly
```

PARAMETERS & ARGUMENTS

PYTHON OFFERS FLEXIBLE WAYS TO PASS ARGUMENTS TO FUNCTIONS.

```
# Positional arguments – matched by position
def power(base, exp):
    return base ** exp

power(2, 8)          # base=2, exp=8

# Keyword arguments – matched by name, order doesn't matter
power(exp=8, base=2)
```

When defining the API of a function, you can constrain how arguments can be passed:

```
# positional-only parameters (before /) – cannot be passed as keywords
# keyword-only parameters (after *) – cannot be passed positionally
# normal parameters (between / and *) can be passed either way
def f(pos_only, /, normal, *, kw_only):
    pass

f(1, 2, kw_only=3)    # valid
f(pos_only=1, normal=2, kw_only=3)  # invalid – pos_only is positional-only
```

DEFAULT PARAMETER VALUES

Default parameter values — used when the argument is omitted

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"
```

```
greet("Alice")           # 'Hello, Alice!'
greet("Bob", "Hi")       # 'Hi, Bob!'
```

Mutable default arguments are a classic trap!

✗ Do NOT use mutable objects as default values!

```
# the default list is shared across all calls
# that omit the 'lst' argument
def append_to(val, lst=[]):
    lst.append(val)
    return lst
```

```
append_to(1)  # [1]
append_to(2)  # [1, 2] ← not [2]!
```

✓ Use `None` and create a new object inside the function.

```
def append_to(val, lst=None):
    if lst is None:
        lst = []
    lst.append(val)
    return lst
```

```
append_to(1)  # [1]
append_to(2)  # [2] ← correct!
```



`*args` **AND** `**kwargs`

COLLECT AN ARBITRARY NUMBER OF POSITIONAL OR KEYWORD ARGUMENTS.

```
def total(x, *args): # *args - collects extra positional arguments into a tuple
    return sum(args) - x

total(1, 2, 3, 4)    # (2 + 3 + 4) - 1 = 8

def display(prefix, **kwargs): # **kwargs - collects extra keyword arguments into a dict
    for key, value in kwargs.items():
        print(f"{prefix} → {key} = {value}")

display("INFO", name="Alice", score=9.5) # INFO → name = Alice \n INFO → score = 9.5
```

Unpacking when calling — the reverse: spread a collection into individual arguments

```
params = (2, 8)
power(*params)           # same as power(2, 8)

options = {"greeting": "Hi", "name": "Bob"}
greet(**options)         # same as greet(name="Bob", greeting="Hi")
```



LAMBDA FUNCTIONS

lambda **CREATES A SMALL ANONYMOUS FUNCTION - A SINGLE EXPRESSION, NO STATEMENTS.**

```
# Equivalent definitions
def square(x):
    return x ** 2

square = lambda x: x ** 2  # same thing
```

Common use: Inline callbacks (e.g., for sorting)

```
students = [("Alice", 9.5), ("Bob", 7.0), ("Carol", 8.5)]

# Sort by score descending
students.sort(key=lambda s: s[1], reverse=True)
```

Lambdas cannot contain assignments, `if / else` *statements*, or multi-line logic. Use `def` whenever the logic is non-trivial.

NESTED FUNCTIONS & CLOSURES

FUNCTIONS CAN BE DEFINED INSIDE OTHER FUNCTIONS AND CAN CAPTURE VARIABLES FROM THEIR ENCLOSING SCOPE.

```
def make_multiplier(factor):  
    def multiply(x):  
        return x * factor    # 'factor' is captured from the enclosing scope  
    return multiply  
  
double = make_multiplier(2)  
triple = make_multiplier(3)  
  
double(5)    # 10  
triple(5)    # 15
```

Each call to `make_multiplier` creates a **new** closure with its own `factor` .

Note: Functions are first-class objects, so you can return them, pass them as arguments, and assign them to variables.

SCOPE: THE LEGB RULE

PYTHON RESOLVES VARIABLE NAMES BY SEARCHING FOUR SCOPES IN ORDER:

1. Local — Inside the current function
2. Enclosing — Enclosing function (like in closures!)
3. Global — Module top level
4. Built-in — Python's defaults (`len` , `print`)

```
x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x) # 1. local
    inner()
    print(x)     # 2. enclosing

outer()
print(x)        # 3. global
```

MODIFYING SCOPE VARIABLES

Use `global` to modify a variable in the global scope:

```
count = 0
def set_to_ten():
    count = 10 # create

set_to_ten()
print(count) # 0
```

```
count = 0
def set_to_ten():
    global count # reference the global variable
    count = 10

set_to_ten()
print(count) # 10
```

Use `nonlocal` to modify a variable in the enclosing scope:

```
def outer():
    count = 0
    def inner():
        count = 10 # create
    inner()
    print(count) # 0
outer()
```

```
def outer():
    count = 0
    def inner():
        nonlocal count
        count = 10
    inner()
    print(count) # 10
outer()
```

DECORATORS

A DECORATOR IS A FUNCTION THAT TAKES ANOTHER FUNCTION AND EXTENDS ITS BEHAVIOR WITHOUT MODIFYING IT.

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"Executed in {time.time() - start:.4f}s")
        return result
    return wrapper
```

Applied using the `@` syntax:

```
@timer
def train_model(x):
    time.sleep(x) # simulate work
    return "Model Trained!"

train_model(1) # prints: Executed in 1.0002s
```



TYPE HINTS

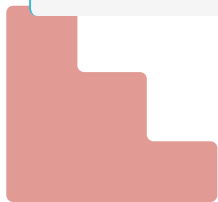
WHILE PYTHON IS DYNAMICALLY TYPED, YOU CAN OPTIONALLY ADD TYPE HINTS FOR BETTER READABILITY AND TOOLING SUPPORT.

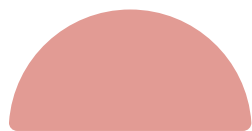
```
def greet(name: str, greeting: str = "Hello") → str:
    return f"{greeting}, {name}!"

greet("Alice")           # 'Hello, Alice!'
greet("Bob", "Hi")       # 'Hi, Bob!'
greet(42)                # still works at runtime, but type checkers will warn
```

Type hints are not enforced at runtime but can be checked by static analysis tools like `mypy` or used by IDEs for autocompletion and error detection.

Personal opinion: Type hints are a great way to document your code and catch bugs early, especially in larger codebases. They can be a bit verbose for simple functions, but the benefits often outweigh the costs. Make a favor to your future self and your teammates: use type hints!





CLASSES



DEFINING A CLASS

A CLASS CONSISTS OF ATTRIBUTES (DATA) AND METHODS (FUNCTIONS) THAT OPERATE ON THAT DATA.

Define a class using the `class` keyword.

```
class Dog:
    # __init__ initialises the instance's state
    # similar to a constructor in other languages
    def __init__(self, name, age):
        self.name = name
        self.age = age

    # instance method
    # self refers to the instance calling the method
    def bark(self):
        return f"{self.name} says: Woof!"
```

```
# create an instance of Dog
rex = Dog("Rex", 5)
luna = Dog("Luna", 3)

# call the bark method
print(rex.bark()) # Rex says: Woof!
print(luna.bark()) # Luna says: Woof!
# this is what happens under the hood:
print(Dog.bark(luna))

# access attributes
print(rex.name) # Rex
print(luna.age) # 3

# modify attributes
rex.age += 1
print(rex.age) # 6
```

`__init__` **AND** `__new__`

THE `__init__` METHOD IS USED TO INITIALIZE AN INSTANCE AFTER IT'S BEEN CREATED. THE ACTUAL CREATION HAPPENS IN THE `__new__` METHOD.

```
class MyClass:
    def __new__(cls, *args, **kwargs):
        print("Creating instance ... ")
        instance = super().__new__(cls) # allocates memory
        return instance # you could return a different object here if you wanted

    def __init__(self, value):
        print("Initializing instance ... ")
        self.value = value

obj = MyClass(42)
# Output:
# Creating instance ...
# Initializing instance ...
```

Note: It is rare to override `__new__` unless you need to control the creation of instances (e.g., for singletons, immutable types, or metaclasses). Most of the time, you'll just implement `__init__`.

INSTANCE VS CLASS ATTRIBUTES

ATTRIBUTES CAN BELONG TO THE OBJECT (INSTANCE) OR TO THE CLASS ITSELF.

```
class Circle:
    pi = 3.14159 # class attribute

    def __init__(self, radius):
        self.radius = radius # instance attribute

    def area(self):
        return Circle.pi * self.radius ** 2
```

```
c1 = Circle(5)
c2 = Circle(10)

print(c1.radius) # 5
print(c2.radius) # 10
print(Circle.pi) # 3.14159
print(c1.pi) # 3.14159 (looked up from class)
print(Circle.radius) # AttributeError

# modify class attribute
Circle.pi = 2
print(c1.area()) # 50 (uses updated pi)
print(c2.area()) # 200 (also uses updated pi)

# instance attributes can shadow class attributes
c1.pi = 4 # creates new instance attribute
print(c1.pi) # 4
print(c2.pi) # 2 (still looks up from class)
```

INSTANCE VS CLASS VS STATIC METHODS

PYTHON SUPPORTS THREE TYPES OF METHODS WITHIN A CLASS.

```
class Circle:
    # instance method (has access to the instance)
    def area(self): ...

    # class method (has access to the class)
    @classmethod
    def unit(cls): return cls(1)

    # static method (no direct access)
    @staticmethod
    def is_valid_radius(r):
        # you can still access class attributes
        # as Circle.{attr}, but no self or cls
        return r > 0
```

```
c = Circle(5)

# instance method – needs an instance
print(c.area())
print(Circle.area(c))

# class method
print(Circle.unit()) # Circle(radius=1)
print(c.unit())      # also works (avoid)

# static method
print(Circle.is_valid_radius(5)) # True
print(c.is_valid_radius(-1))    # False (avoid)
```

Usually, `@classmethod` is used for alternative constructors, and `@staticmethod` is used for utility functions related to the class but not needing access to instance or class data.

ACCESS MODIFIERS

PYTHON DOESN'T HAVE TRUE ACCESS MODIFIERS LIKE `private` OR `protected`, BUT IT USES NAMING CONVENTIONS TO INDICATE INTENDED VISIBILITY.

```
class Model:
    def __init__(self):
        self.public = "Anyone can read/write"

        # protected: single underscore by convention
        # meant for internal/subclass use
        # still accessible
        self._protected = "Please don't touch"

        # private: double underscore
        # name mangled to prevent accidental access
        # still accessible with _ClassName__attr
        self.__private = "Strictly internal"
```

These are just conventions and do not enforce access restrictions.

They are used to signal intent to other developers.

```
m = Model()

print(m.public)           # anyone can read/write
print(m._protected)      # handle with care
print(m.__private)       # AttributeError
print(m._Model__private) # name mangled
```

PROPERTIES

USE `@property` TO DEFINE COMPUTED ATTRIBUTES THAT LOOK LIKE REGULAR ATTRIBUTES.

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self): # the getter
        return self._radius

    @radius.setter
    def radius(self, value): # the setter
        if value < 0:
            raise ValueError("Radius must be ≥ 0")
        self._radius = value
```

Properties let you write code that *looks* like simple attribute access, but executes functions under the hood.

```
c = Circle(5)

# calls the getter
print(c.radius) # 5

# calls the setter
c.radius = 10

# triggers the ValueError in the setter
c.radius = -2
```

Rule of thumb: While you are allowed to perform complex computations in a property, it's best to keep them efficient since the caller expects attribute-like access. See `cached_property` for expensive computations that should be cached.

INHERITANCE

INHERITANCE ALLOWS YOU TO CREATE A NEW CLASS THAT INHERITS ATTRIBUTES AND METHODS FROM AN EXISTING CLASS BUT CAN ALSO OVERRIDE OR EXTEND THEM.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self): raise NotImplementedError

class Dog(Animal):
    def __init__(self, name, breed):
        # use super().{method} to call
        # the parent class's method
        super().__init__(name)
        self.breed = breed

    def speak(self): return f"{self.name} barks!"

class Cat(Animal):
    # __init__ is inherited from Animal

    def speak(self): return f"{self.name} meows!"
```

Inheritance allows *polymorphism*, where you can treat different subclasses as instances of the parent class and call methods that behave differently based on the actual subclass.

```
animals =[
    Dog("Rex", "German Shepherd"),
    Cat("Luna")
]

for a in animals:
    print(a.speak())

# Output:
# Rex barks!
# Luna meows!
```

MULTIPLE INHERITANCE

PYTHON SUPPORTS MULTIPLE INHERITANCE, WHERE A CLASS CAN INHERIT FROM MORE THAN ONE PARENT CLASS.

```
class Flyer:
    def fly(self):
        return "I can fly!"

    def talk(self):
        return "Flyers can talk too!"

class Swimmer:
    def swim(self):
        return "I can swim!"

    def talk(self):
        return "Swimmers can talk too!"

class Duck(Flyer, Swimmer):
    pass
```

```
d = Duck()
print(d.fly())    # I can fly!
print(d.swim())   # I can swim!
```

When methods are defined in multiple parent classes, Python uses the **Method Resolution Order (MRO)** to determine which method to call.

```
print(d.talk())   # Flyers can talk too!

# mro goes depth-first, left-to-right
print(Duck.mro())
# [<class '__main__.Duck'>,
#  <class '__main__.Flyer'>,
#  <class '__main__.Swimmer'>,
#  <class 'object'>]
```

ABSTRACT BASE CLASSES

SOMETIMES YOU WANT A PARENT CLASS TO *ENFORCE* THAT SUBCLASSES IMPLEMENT SPECIFIC METHODS.

```
from abc import ABC, abstractmethod

class Animal(ABC):
    def __init__(self, name):
        self.name = name

    @abstractmethod
    def speak(self):
        pass # To be overridden by subclasses

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed

    def speak(self):
        return f"{self.name} barks!"
```

```
# you cannot instantiate an abstract class
# that has unimplemented abstract methods
animal = Animal("Generic") # TypeError
dog = Dog("Rex", "German Shepherd")
print(dog.speak()) # Rex barks!
```

`abc.ABC` forces nominal subtyping, meaning that a class must explicitly inherit from the abstract base class to be considered a subclass, even if it implements all the required methods.

This is in contrast to python's usual duck typing, where an object is considered a subtype if it has the necessary methods/attributes, regardless of its actual class. If you care about type hints, you can use `typing.Protocol` for structural subtyping.

DUNDER/MAGIC METHODS

DUNDER (DOUBLE-UNDERSCORE) METHODS LET YOUR OBJECTS INTERACT WITH PYTHON'S BUILT-IN OPERATIONS (+, =, len(), print(), ETC.)

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):
        return f"Point({self.x}, {self.y})"

    def __str__(self):
        return f"({self.x}, {self.y})"

    def __add__(self, o):
        return Point(self.x + o.x, self.y + o.y)

    def __eq__(self, o):
        if not isinstance(o, self.__class__):
            return NotImplemented
        return self.x == o.x and self.y == o.y
```

```
p = Point(1, 2)

# printing
print(p)           # (1, 2) - uses __str__
print(repr(p))     # Point(1, 2) - uses __repr__
print(f"{p!r}")    # Point(1, 2) - uses __repr__

# addition
p1 = Point(1, 2)
p2 = Point(3, 4)
print(p1 + p2)     # Point(4, 6)

# equality
p3 = Point(1, 2)
print(p1 == p3)    # True
print(p1 == p2)    # False
print(p1 == (1, 2)) # False - different types
```

COMMON DUNDER METHODS

Method	Triggered by	Purpose
<code>__init__</code>	<code>MyClass(...)</code>	Initialise the instance
<code>__repr__</code>	<code>repr(obj)</code> , REPL display	Unambiguous string representation
<code>__str__</code>	<code>str(obj)</code> , <code>print(obj)</code>	Human-readable string
<code>__len__</code>	<code>len(obj)</code>	Return the object's "length"
<code>__add__</code>	<code>obj + other</code>	Addition
<code>__eq__</code>	<code>obj == other</code>	Equality comparison
<code>__lt__</code>	<code>obj < other</code>	Less-than (enables sorting)

MORE DUNDER METHODS

Method	Triggered by	Purpose
<code>__getitem__</code>	<code>obj[key]</code>	Indexing / key lookup
<code>__iter__</code>	<code>for x in obj</code>	Return an iterator
<code>__contains__</code>	<code>x in obj</code>	Membership test
<code>__call__</code>	<code>obj(...)</code>	Make the instance callable

And many more for arithmetic, context managers, attribute access, etc. See the [Python data model docs](#) for the full list.

You don't need to implement all of these. Just the ones that make sense for your class. For example, a `Point` might implement `__add__` and `__eq__`, but not `__len__` or `__call__`.

DATACLASSES (PYTHON 3.7+)

`@dataclass` **AUTO-GENERATES BOILERPLATE METHODS LIKE `__init__`, `__repr__`, AND `__eq__` FOR CLASSES THAT PRIMARILY HOLD DATA.**

```
from dataclasses import dataclass, field

@dataclass
class Student:
    name: str
    # default values for fields
    score: float = 30
    # use default_factory for mutable types
    grades: list[float] = field(
        default_factory=list)

    def __post_init__(self):
        # runs after the auto-generated __init__
        if not (0 ≤ self.score ≤ 30):
            raise ValueError
```

```
# __init__ is generated automatically
alice = Student("Alice", 30)
bob   = Student("Bob", 7.0, [7.0, 8.0])
charlie = Student("Charlie")

# __repr__ is generated automatically
print(alice)
# Student(name='Alice', score=9.5, grades=[])
print(bob)
# Student(name='Bob', score=7.0, grades=[7.0, 8.0])
print(charlie)
# Student(name='Charlie', score=30, grades=[])

# __eq__ is generated (compares all fields)
alice == bob # False
charlie == Student("Charlie") # True
```



MORE DATACLASS FEATURES

THE BEHAVIOR OF DATACLASSES CAN BE CUSTOMIZED WITH PARAMETERS TO THE `@dataclass` DECORATOR. SEE THE DATACLASSES DOCS FOR DETAILS.



Parameter	Description
<code>init</code>	If <code>False</code> , don't generate <code>__init__</code>
<code>kw_only</code>	If <code>True</code> , make all fields keyword-only in the generated <code>__init__</code>
<code>repr</code>	If <code>False</code> , don't generate <code>__repr__</code>
<code>eq</code>	If <code>False</code> , don't generate <code>__eq__</code>
<code>order</code>	If <code>True</code> , generate ordering methods (<code>__lt__</code> , <code>__gt__</code> , etc.)
<code>frozen</code>	If <code>True</code> , make instances immutable (no attribute assignment)



METACLASSES

IN PYTHON, CLASSES THEMSELVES ARE OBJECTS. JUST AS AN OBJECT IS AN INSTANCE OF A CLASS, A CLASS IS AN INSTANCE OF A METACLASS.

```
# type is the default metaclass for all classes
class RegistryMeta(type):
    models = {}

    # called when the class is created
    def __new__(mcs, name, bases, attrs):
        # here you could modify the class
        # (add attributes, enforce rules, etc.)
        new_class = super().__new__(
            mcs, name, bases, attrs
        )
        # register the new class in the registry
        mcs.models[name] = new_class
        return new_class
```

```
class BaseModel(metaclass=RegistryMeta):
    pass

class ResNet(BaseModel):
    pass

class Transformer(BaseModel):
    pass

# we didn't register these manually,
# but the metaclass caught them!
print(RegistryMeta.models.keys())
# dict_keys(['BaseModel', 'ResNet', 'Transformer'])
```

Rule of thumb: If you wonder whether you need metaclasses, you don't. But you will likely read them in library source code!



MODULES & IMPORTS





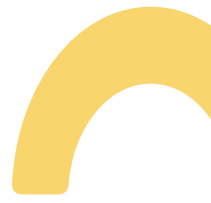
WHY MODULES?

As your program grows, you can't keep everything in one file.

Instead of a 10,000-line file, we split code into logical blocks.

- 
- **Maintainability:** Keep files small and focused.
 - **Reusability:** Share code across projects by importing modules.
 - **Namespacing:** Avoid variable name collisions.

These are the same reasons we use functions — modules are just a higher level of organisation.



WHAT IS A MODULE?

A MODULE IS SIMPLY A `.py` FILE. ANY PYTHON FILE CAN BE IMPORTED BY OTHER FILES.

```
project/  
├─ main.py  
└─ utils.py
```

```
# utils.py  
def add(a, b):  
    return a + b
```

```
PI = 3.14159
```

```
# main.py  
import utils
```

```
PI = 3.14 # This is a different variable – no conflict with utils.PI
```

```
print(utils.add(2, 3)) # 5  
print(utils.PI)       # 3.14159
```

Each module has its own **private namespace** — no accidental clashes between global variables across files.

IMPORT STYLES

1. whole module – access via prefix

```
import math  
math.sqrt(16)                # 4.0
```

2. specific names – no prefix needed

– for items you use frequently

– for items that are unambiguous and unlikely to clash with other names

```
from math import sqrt, pi  
sqrt(16)                     # 4.0
```

3. alias – you access the imported module/item via the alias

– for long module names or conflicting names

– for common conventions (e.g. np for numpy, pd for pandas)

```
import numpy as np  
from sklearn.linear_model import LinearRegression as LR
```

4. everything – you get all (public) items from the module directly in your namespace

– it pollutes the namespace

– it makes it unclear where names come from

```
from math import *  
sqrt(16)                  # 4.0
```

IMPORT MECHANICS

WHEN YOU IMPORT A MODULE, PYTHON:

1. Searches for the file in the directories listed in `sys.path` (builtin modules, current directory, PYTHONPATH, site-packages)
2. Compiles the file to bytecode (if not already cached)
3. Executes it top-to-bottom — **once** — and caches the result in `sys.modules`
4. Binds the module object to the name in the importing namespace

Note: Subsequent imports of the same module return the cached version — they do **not** re-execute the file. Changes to the module's code won't take effect until you restart the interpreter or manually reload the module (see `importlib.reload`).

Important: You should avoid putting side effects (e.g. print statements, file I/O) at the top level of a module, as they will run on import and may cause unexpected behaviour when the module is imported multiple times.

MODULES AS SCRIPTS

YOU CAN RUN A MODULE DIRECTLY AS A SCRIPT:

```
$ python utils.py
```

To make the module both importable and runnable, you can use the `__name__` variable to check if the module is being run directly or imported:

```
# utils.py
def hello(name):
    print(f"Hello, {name}!")

# __name__ is set to "__main__" when the file is run directly, and to the module's name when imported
if __name__ == "__main__":
    hello("world") # without this guard, this would run on import too!
```

This allows you:

- to include (under the guard) test code or command-line interfaces in your modules
- to keep the module importable without side effects when imported by other files.

PACKAGES

A PACKAGE IS A DIRECTORY CONTAINING A (POSSIBLE EMPTY) `__init__.py` FILE. PACKAGES LET YOU ORGANISE RELATED MODULES INTO A NAMESPACE HIERARCHY.

```
myproject/  
├── main.py  
└── ml/  
    ├── __init__.py      # makes 'ml' a package  
    ├── preprocessing.py  
    └── model.py
```

```
# absolute import - always works  
# main.py  
import ml.preprocessing  
from ml.model import LinearRegressor  
  
# relative import - only inside the package  
# ml/preprocessing.py  
from .model import LinearRegressor  # . means "current package"
```

EXPOSING A CLEAN API

`__init__.py` RUNS WHEN THE PACKAGE IS FIRST IMPORTED. USE IT TO EXPOSE A CLEAN PUBLIC API:

```
# ml/__init__.py
from .model import LinearRegressor # re-export the class from the model module
from .preprocessing import normalize

__all__ = ["LinearRegressor", "normalize"] # optional, controls what 'from ml import *' imports
```

Now users import directly from the package, not from internal submodules:

```
# without __init__.py
from ml.model import LinearRegressor

# with __init__.py - cleaner, hides internal structure
from ml import LinearRegressor
```

Exposing a clean API allows you to change the internal structure of your package without breaking users' code, as long as the public API remains consistent.

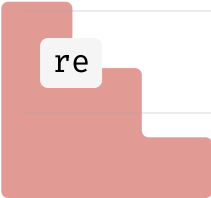


THE STANDARD LIBRARY

PYTHON SHIPS WITH A LARGE STANDARD LIBRARY - NO INSTALLATION NEEDED.



Module	Purpose
<code>os</code> / <code>pathlib</code>	Filesystem, paths, env vars
<code>sys</code>	Interpreter state, <code>sys.argv</code>
<code>math</code>	Trig, log, constants (<code>pi</code> , <code>e</code>)
<code>random</code>	Pseudo-random numbers
<code>json</code>	Serialize / deserialize JSON
<code>re</code>	Regular expressions



Module	Purpose
<code>datetime</code>	Dates, times, timedeltas
<code>collections</code>	<code>Counter</code> , <code>defaultdict</code> , <code>deque</code>
<code>itertools</code>	<code>chain</code> , <code>product</code> , <code>combinations</code>
<code>functools</code>	<code>reduce</code> , <code>partial</code> , <code>lru_cache</code>
<code>subprocess</code>	Run external commands



THIRD-PARTY PACKAGES

PACKAGES NOT IN THE STANDARD LIBRARY ARE DISTRIBUTED VIA **PyPI** AND
INSTALLED WITH `pip` OR `uv`.

```
# install a package
uv pip install requests

# install multiple packages at once
uv pip install numpy pandas matplotlib scikit-learn

# pin exact versions for reproducibility
uv pip install "numpy==2.0.0"

# in project mode, use `uv add` to add dependencies to pyproject.toml
uv add numpy pandas matplotlib scikit-learn
uv add "numpy==2.0.0"
```

Then import and use them in your code:

```
import numpy as np
x = np.array([1, 2, 3])
print(x + 1) # [2 3 4]
```